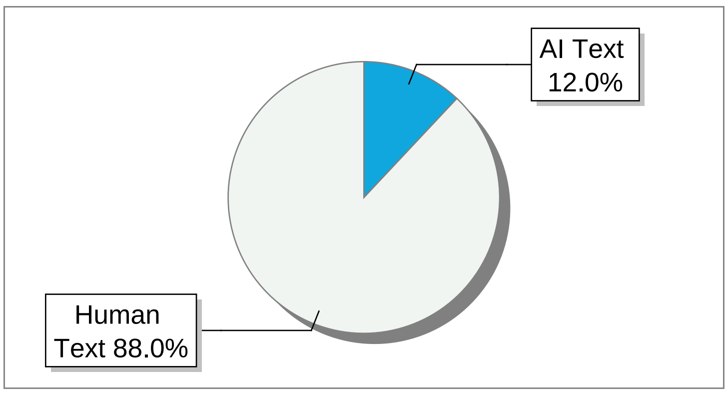
The Report is Generated by DrillBit AI Content Detection Software

Submission Information

Author Name	Ankith
Title	Deep Learning-Based Plant Health and Advisory Platform for Angiosperm Plants
Paper--Submission ID	5653292
Submitted By	lib@pesce.ac.in
Submission Date	2026-05-15
Total [Pages,Sentences,Words]	[47, 483, 8622]
Document type	Research Paper
Language	English

Result Information

AI Text: **12 %**



Disclaimer:

* The content detection system employed here is powered by artificial intelligence (AI) technology.
* Its not always accurate and only help to author identify text that might be prepared by a AI tool.
* It is designed to assist in identifying & moderating content that may violate community guidelines/legal regulations, it may not be perfect.

P.E.S. COLLEGE OF ENGINEERING MANDY - 571 401 (An Autonomous Institution Affiliated to VTU, Belagavi) A Project Report on “Deep Learning-Based Plant Health and Advisory Platform for Angiosperm Plants” Submitted in partial fulfillment for the award of Degree of BACHELOR OF ENGINEERIN IN INFORMATION SCIENCE AND ENGINEERING Submitted by ANKITH M [4PS22IS006] MADAN A [4PS22IS033] MOHITH M Y [4PS23IS405] HEMANTH C N [4PS22IS022] Under the guidance of Dr. MINAVATHI Professor, Dept.
of IS&E, PESCE Mandya.
DEPARTMENT OF INFORMATION SCIENCE AND ENGINEERING P.
E. S.
College of Engineering, Mandya – 571 401 2025-26 PES.
COLLEGE OF ENGINEERING MANDYA-5714 01 (An Autonomous Institution Affiliated to VTU, Belagavi) DEPARTMENT OF INFROMATION SCIENCE AND ENGINEERING This is to certify that, ANKITH M [4PS22IS006], MADAN A [4PS22IS033], MOHITH M Y [4PS22IS405], HEMANTH C N [4PS22IS022], has been successfully completed the Project work “Deep Learning- -Based Plant Health and Advisory Platform for Angiosperm Plants” in partial fulfillment for the award of Bachelor of Engineering of PES College of Engineering, Mandya (An Autonomous Institution, Affiliated to VTU, Belagavi) during the year 2025-26.
It is certified that all corrections indicated in internal assessment have been incorporated in the report deposited in the library.
The Project Work have been approved as it satisfies the academic requirements in respect of work prescribed for the degree in Bachelor of Engineering in Information Science and Engineering discipline.
Signature of Guide Signature of HOD Dr.
Minavathi Dr.
T S Prabhakar Professor, Professor & Head, Dept.
of ISE, Dept.
of ISE, PESCE, Mandya.
PESCE, Mandya.
SL No.
Name of the Examiner and Address Signature with Date 1.
2. Committee Members DECLARATION We ANKITH M [4PS22IS006], MADAN A [4PS22IS033], MOHITH M Y [4PS22IS405], HEMANTH C N [4PS22IS022], students of 8th semester of Bachelor of Engineering in Information Science & Engineering at P.E.S.
College of Engineering, Mandya, hereby declare that the work being presented in the Project Work entitled “Deep Learning- -Based Plant Health and Advisory Platform for Angiosperm Plants” is an authentic record of the work that has been independently carried out by us and submitted in partial fulfilment of the requirement for the award of the degree of Bachelor of Engineering in Information Science & Engineering at P.E.S.
College of Engineering during the academic year 2025-2026.
The work contained in this report has not been submitted to any other university or institution or professional body for the award of any other degree or fellowship.
Date : ANKITH M [4PS22IS006] Place : Mandya MADAN A [4PS22IS033] MOHITH M Y [4PS23IS405] HEMANTH C N [4PS22IS022] ACKNOWLEDGEMENT The satisfaction that accompanies the successful completion of this project would be incomplete without expressing our gratitude to the Almighty God and the people who made this work possible.
We are grateful to P.
E. S.
College of Engineering, Mandya for providing us an opportunity to enhance our knowledge through this project work.
We express our sincere thanks to Dr.
N. L.
Murali Krishna, Principal, PESCE Mandya, for providing us the opportunity and facilities to carry out and present this project work.
We express our heartfelt thanks to Dr.

Prabhakar TS, Professor and Head, Department of Information Science and Engineering, PESCE Mandya, for her encouragement and support throughout our project work.
Her cooperation and guidance helped in the successful completion of this project report.
We would also like to express our profound thanks to our project guide Dr.
Minavathi, Professor, Department of Information Science and Engineering, for her keen interest, valuable guidance, and constant encouragement during the entire project process.

ANKITH M [4PS22IS006] MADAN A [4PS22IS033] MOHITH M Y [4PS23IS405] HEMANTH C N [4PS22IS022] ABSTRACT Agriculture is a critical sector for global food security and economic growth, particularly in countries like India where a majority of the population depends on farming for livelihood.
However, plant diseases continue to pose a major threat to crop productivity, quality, and profitability.
Early identification of plant diseases is essential, yet traditional diagnosis methods rely on expert observation, which is time-consuming, subjective, and often inaccessible to small- scale farmers in remote regions.
The lack of timely disease detection and relevant advisory support results in severe crop losses and reduced profitability.
To address these challenges, this project proposes an integrated AI-based Plant Health and Advisory Platform developed specifically for angiosperm plants.
The system employs a deep learning-based Convolutional Neural Network (CNN) model for automated disease prediction using leaf images captured through a smartphone or camera.
The CNN architecture extracts visual features such as lesions, texture variations, and color deviations to classify diseases with high accuracy.
The dataset is preprocessed through resizing, augmentation, and normalization to ensure model robustness and generalization.
The trained model produces disease identification results along with confidence percentages, enabling farmers to make informed decisions.
In addition to disease detection, the platform integrates an intelligent multilingual advisory system and the LLaMA 3.2 language model.
This enables real-time voice-based communication where farmers can ask queries and receive treatment suggestions, pesticide recommendations, and preventive measures in their native language.
The system also incorporates weather forecasting and soil analytics to provide localized crop management insights.
Experimental evaluation demonstrates that the CNN model achieves high accuracy in classifying healthy and diseased leaf samples, proving its effectiveness in real-world agricultural environments.
The proposed solution aims to enhance precision farming, reduce crop loss, and provide affordable AI support accessible to farmers nationwide.
The platform is scalable, user- friendly, and capable of transforming agricultural diagnostic and advisory processes.

TABLE OF CONTENTS

LIST OF FIGURES

CHAPTER 1
INTRODUCTION

Agriculture plays a vital role in ensuring food security and economic stability, particularly in countries like India where a large portion of the population depends on farming for livelihood.
However, plant diseases remain a major challenge in agriculture, leading to reduced crop yield, poor quality produce, and significant financial losses for farmers.
Early and accurate detection of plant diseases is essential to prevent large-scale damage and to ensure sustainable agricultural practices.
In traditional agricultural practices, disease identification is performed manually by farmers or agricultural experts based on visual inspection of plant leaves.
This approach is often time- consuming, subjective, and prone to errors, especially in rural areas where access to expert knowledge is limited.
Additionally, many farmers lack awareness of proper treatment methods, resulting in delayed or incorrect interventions.
With the advancement of Artificial Intelligence (AI) and Deep Learning technologies, automated plant disease detection systems have emerged as efficient solutions.
Convolutional Neural Networks (CNNs) have shown significant success in image classification tasks, including plant disease recognition, by extracting complex visual features from leaf images.

These systems enable faster and more accurate diagnosis compared to manual methods.
However, most existing solutions focus only on disease classification and lack comprehensive advisory support.
They do not provide complete guidance such as treatment recommendations, preventive measures, or real-time interaction with users.
Furthermore, many systems depend entirely on internet connectivity and support limited languages, making them less accessible to farmers in rural regions.

To address these limitations, this project proposes a Deep Learning-Based Plant Health and Advisory Platform for Angiosperm Plants, designed as an integrated AI-powered system.
The platform combines plant disease detection, crop recommendation, and intelligent advisory services into a unified full-stack application.
DEPT. OF ISE, PESCE, MANDYA 2 DEEP LEARNING-BASED PLANT HEALTH AND ADVISORY PLATFORM FOR ANGIOSPERM PLANTS 2025-26 The system utilizes a CNN model trained on the PlantVillage dataset to classify plant diseases from leaf images.
In addition, it incorporates a dual-mode AI advisory system using cloud- based models (Gemini API) and local models (Ollama – LLaMA 3.2), ensuring continuous operation even in low-connectivity environments.
A key feature of the proposed system is its multilingual capability, allowing users to interact with the system in multiple Indian languages.
The platform also supports chatbot-based interaction and optional WhatsApp integration, enabling real-time assistance for farmers.
The system is developed using modern web technologies, including Next.js for frontend and backend integration, MongoDB for data storage, and RESTful APIs for communication between modules.
By combining deep learning, artificial intelligence, and full-stack development, the proposed system provides an efficient, scalable, and user-friendly solution for modern agricultural challenges.
With the growing adoption of AI in agriculture, intelligent advisory platforms like this are expected to significantly improve productivity, reduce losses, and empower farmers with data- driven decision-making tools.
1.1. AIM The aim of this project is to develop a Deep Learning-Based Plant Health and Advisory Platform for Angiosperm Plants that enables accurate and early detection of plant diseases using image-based analysis and provides intelligent, real-time guidance to farmers.
The system focuses on utilizing a Convolutional Neural Network (CNN) model to classify plant diseases from leaf images and generate reliable diagnostic outputs.
In addition to disease detection, the platform aims to integrate an AI-based advisory system capable of providing treatment suggestions, preventive measures, and crop recommendations based on user inputs.
Another key objective of the system is to implement a dual-mode artificial intelligence framework, where cloud-based models (Gemini API) are used for advanced reasoning, and locally deployed models (Ollama – LLaMA 3.2) ensure continuous operation in offline or low- connectivity environments.
DEPT. OF ISE, PESCE, MANDYA 3 DEEP LEARNING-BASED PLANT HEALTH AND ADVISORY PLATFORM FOR ANGIOSPERM PLANTS 2025-26 The project also aims to enhance accessibility by supporting multilingual interaction, allowing farmers to communicate with the system in multiple Indian languages.
Furthermore, the system is designed as a full-stack web application with database integration to store user data, analysis results, and interaction history.
Overall, the aim of this project is to provide an intelligent, scalable, and user-friendly agricultural solution that reduces dependency on manual diagnosis, improves decision- making, and supports sustainable farming practices through the effective use of artificial intelligence and deep learning technologies.

1.2 OBJECTIVES
The project aims to address the limitations of traditional agricultural practices by developing an intelligent and integrated system for plant health monitoring and advisory support.
To achieve this, the work is structured into four core objectives: Objective 1: Integrated AI Platform To design and develop a unified AI-powered agricultural platform that integrates plant disease detection, crop advisory,

chatbot interaction, and data storage into a single full-stack web application.
The system aims to provide a seamless user experience through a centralized interface that supports real-time interaction and maintains historical records for analysis.
Objective 2: Deep Learning-Based Disease Detection To develop and deploy a Convolutional Neural Network (CNN) model trained on the PlantVillage dataset (~87,000 images across 38 classes) for accurate classification of plant diseases from leaf images.
The objective is to enable early detection and improve diagnostic accuracy using image-based deep learning techniques.
Objective 3: Real-Time and Offline AI Support To implement a dual-mode artificial intelligence system that combines cloud-based models (Gemini API) with a locally hosted model (Ollama – LLaMA 3.2).
This ensures continuous DEPT.
OF ISE, PESCE, MANDYA 4 DEEP LEARNING-BASED PLANT HEALTH AND ADVISORY PLATFORM FOR ANGIOSPERM PLANTS 2025-26 system functionality by providing real-time responses when internet connectivity is available and fallback support in offline environments.
Objective 4: Multilingual Accessibility To enable multilingual interaction by supporting multiple Indian languages, allowing users to communicate with the system in their preferred language.
This objective aims to improve accessibility and usability, especially for farmers in rural areas, through AI-generated localized responses.

1.3 EXISTING SYSTEM

In traditional agricultural practices, plant disease detection is primarily performed through manual observation by farmers or consultation with agricultural experts.
Farmers typically identify diseases based on visible symptoms such as discoloration, spots, or deformities in leaves.
However, this method is highly dependent on individual experience and knowledge, which may lead to incorrect diagnosis and delayed treatment.
In many rural areas, access to agricultural experts or extension services is limited, making it difficult for farmers to obtain timely guidance.
As a result, diseases often spread before appropriate action is taken, leading to reduced crop yield and financial loss.
Additionally, traditional methods do not provide standardized or consistent results, as the diagnosis varies from person to person.
Several digital solutions have been developed to assist farmers, including mobile applications and web-based platforms for plant disease identification.
Most of these systems use image processing or basic machine learning techniques to classify diseases.
While these solutions provide some level of automation, they are often limited in scope and functionality.
One of the major limitations of existing systems is that they focus only on disease detection without offering comprehensive advisory support such as treatment recommendations, preventive measures, or crop suggestions.
Moreover, many of these systems require continuous internet connectivity, which restricts their usability in remote or low-connectivity regions.

DEPT. OF ISE, PESCE, MANDYA 5 DEEP LEARNING-BASED PLANT HEALTH AND ADVISORY PLATFORM FOR ANGIOSPERM PLANTS 2025-26 Another significant drawback is the lack of multilingual support, as many applications are designed primarily in English, making them difficult to use for farmers who are more comfortable with regional languages.
Additionally, existing systems generally do not provide conversational interfaces, reducing user interaction and accessibility.
Furthermore, most current solutions do not integrate multiple functionalities into a single platform.
They operate as standalone systems, lacking features such as chatbot interaction, real-time advisory, database storage, and user history tracking.
Due to these limitations, existing systems are not fully effective in addressing the real-world challenges faced by

farmers.
There is a need for a more comprehensive, intelligent, and accessible solution that integrates disease detection, advisory services, and multilingual interaction within a unified platform.

1.4 PROPOSED SYSTEM

To overcome the limitations of existing systems, this project proposes a Deep Learning-Based Plant Health and Advisory Platform for Angiosperm Plants, named Kisan Sahayak.
The system is designed as an integrated, AI-powered solution that combines plant disease detection, intelligent advisory services, and user interaction within a unified full-stack platform.
The core component of the proposed system is a Convolutional Neural Network (CNN) model trained on the PlantVillage dataset, which contains approximately 87,000 labeled images across 38 plant disease classes.
The model is capable of accurately classifying plant diseases based on leaf images provided by the user, enabling early detection and reducing dependency on manual inspection.
Beyond disease detection, the system incorporates an AI-based assistance module that generates detailed insights, including causes of diseases, treatment methods, and preventive measures.
This module leverages Large Language Models (LLMs) to provide context-aware and informative responses, supporting better agricultural decision-making.
  A key feature of the proposed system is the implementation of a dual-mode AI architecture.
The system uses the Gemini API for cloud-based intelligence when internet connectivity is available.
In offline or low-connectivity environments, it switches to a locally hosted Ollama model based on LLaMA 3.2.
This ensures continuous system operation and makes the platform suitable for deployment in rural areas.
The platform also includes a multilingual interaction system, allowing users to communicate in multiple Indian languages.
This enhances accessibility and usability, especially for farmers who prefer regional languages.
The system supports conversational interaction, enabling users to query agricultural problems and receive real-time AI-driven assistance.
In addition, the system provides crop recommendation and advisory support based on user inputs such as location, soil type, and seasonal factors.

This helps farmers make informed decisions regarding crop selection and management practices.
The entire system is developed as a full-stack web application using modern technologies.
The frontend and backend are implemented using Next.js, while MongoDB is used for data storage.
RESTful APIs are used to manage communication between modules, ensuring efficient data processing and scalability.
Overall, the proposed system delivers a comprehensive, intelligent, and accessible solution by integrating deep learning, artificial intelligence, and modern web technologies.
It aims to improve crop health monitoring, reduce agricultural losses, and empower farmers with data- driven decision-making tools.

CHAPTER 2

LITERATURE SURVEY Plant disease detection and agricultural advisory systems have gained significant attention due to advancements in Artificial Intelligence (AI) and Deep Learning.
Traditional methods rely on manual inspection, which is often inaccurate and time-consuming.
To overcome these challenges, researchers have proposed various machine learning and deep learning techniques for automated disease detection and advisory systems.
[1] A.
Y. Ashurov et al., “Deep Learning-Based Plant Disease Detection,” 2025 This study proposes an advanced CNN

model integrated with squeeze-and-excitation blocks for efficient plant disease detection.
The model improves feature extraction and enhances classification accuracy.
The approach contributes to better crop protection and yield optimization.
[2] D.
Devarajan et al., “AI-Based Real-Time Disease Diagnosis in Plants,” 2026 This research introduces an enhanced CNN (E-CNN) architecture for detecting plant diseases in crops such as apple, corn, and potato.
The model achieves an accuracy of approximately 98%, demonstrating the effectiveness of deep learning in early disease identification.
[3] K.
Yasin et al., “Review of Deep Learning Architectures for Plant Disease Detection,” 2025 This paper provides a comprehensive review of deep learning models used between 2018 and 2024.
It highlights various CNN architectures and discusses challenges such as dataset limitations and real-world deployment issues.
[4] V.
Sharma et al., “Distributed CNN-Based Plant Disease Detection,” 2024 This work combines CNN with distributed computing techniques to improve large-scale disease detection.
The system achieves around 95% accuracy and provides a user-friendly interface for farmers, improving accessibility.
 [5] P.
Rastogi et al., “Early Disease Detection in Plants Using CNN,” 2024 This study focuses on early detection of plant diseases using CNN models trained on PlantVillage datasets.
The approach improves detection accuracy and contributes to reducing crop losses through early intervention.
[6] M.
Shafay et al., “Recent Advances in Plant Disease Detection,” 2025 This review analyzes modern deep learning techniques, including RGB and hyperspectral imaging, for disease detection.
It emphasizes the importance of automated systems in improving agricultural productivity.
[7] M.
El Sakka et al., “CNN Applications in Smart Agriculture,” 2025 This paper reviews the role of CNN in agricultural applications such as disease detection, crop classification, and yield prediction.
It highlights CNN as a key technology for smart farming systems.
[8] M.
S. Krishna et al., “Multi-Dataset Deep Learning for Plant Disease Detection,” 2025 This research uses multiple datasets and CNN architectures such as EfficientNet and ResNet to improve generalization.
The study demonstrates strong performance across different environmental conditions.
[9] C.
Pal et al., “Lightweight and Explainable CNN Model for Plant Disease Detection,” 2025 This paper proposes a lightweight deep learning model combining MobileNetV2 with residual connections.
It improves interpretability and reduces computational complexity, making it suitable for real-world deployment.
[10] Suri et al., “CNN-Based Real-Time Crop Disease Classification System,” 2025 This study presents a CNN-based system for real-time disease classification with an integrated treatment recommendation module.
The system supports farmers in making informed decisions and improves agricultural productivity.

CHAPTER 3
SYSTEM REQUIREMENT AND ANALYSIS

3.1 SOFTWARE REQUIREMENTS

- Operating System: The system should be compatible with Windows 10 or higher and Linux-based operating systems such as Ubuntu.
This ensures flexibility in development and deployment environments and supports smooth execution of both AI

models and web applications.
- Programming Language: The system is developed using Python for machine learning and model development, and JavaScript/TypeScript for full-stack web application development.
Python provides efficient support for data preprocessing and deep learning, while JavaScript enables dynamic frontend and backend functionality.
- Development Environment: Development tools such as Visual Studio Code and Jupyter Notebook are used for coding, debugging, and testing.
These environments provide an integrated workspace for both machine learning experimentation and full-stack development.
- Deep Learning Framework: The CNN model is implemented using TensorFlow/Keras, which provides efficient tools for building, training, and evaluating deep learning models for plant disease detection.
- AI Integration: The system integrates cloud-based AI using Gemini API and local AI inference using Ollama (LLaMA 3.2).
This ensures continuous operation in both online and offline environments.
- Frontend Framework: The user interface is developed using Next.js, which supports both frontend rendering and backend API handling in a unified framework.
- Backend and API: The backend is implemented using Next.js API routes, which enable communication between the frontend, AI modules, and database through RESTful services.
- Database: MongoDB is used for storing user data, disease detection results, chatbot interactions, and system logs.
It provides flexible schema design and efficient data management.
- Version Control: The project can be managed using Git and GitHub for version tracking, collaboration, and backup of source code.

3.2 HARDWARE REQUIREMENTS

- Processor: The system requires a minimum Intel Core i3 processor or equivalent for basic operation.
For better performance during model execution and system processing, an Intel Core i5/i7 or AMD equivalent is recommended.

- RAM: A minimum of 4 GB RAM is required for basic execution of the system.
For smoother multitasking and better performance, 8 GB RAM or higher is recommended.
- Storage: At least 10–20 GB of storage is required to store datasets, trained models, application files, and logs.
An SSD is preferred for faster read/write operations and improved performance.
- GPU: A GPU is not mandatory but recommended for faster training and execution of deep learning models.
- Internet Connectivity: Internet access is required for cloud-based AI services.
However, the system can also operate in offline mode using locally deployed AI models.

CHAPTER 4
OBJECTIVE 1: INTEGRATED AI PLATFORM

4.1 INTRODUCTION

The primary objective of this chapter is to design and implement an integrated AI-based platform that combines plant disease detection, AI assistance, and user interaction into a unified system.
The system is designed to provide real-time analysis and intelligent recommendations to users through a seamless interface.
By integrating deep learning models, AI modules, and database systems within a full-stack architecture, the platform ensures efficient data flow, scalability, and ease of use.
This objective focuses on building the foundational structure that enables all subsequent functionalities of the

system.

4.2 KEY CONTRIBUTIONS

4.2.1 Unified Platform Development: The system integrates multiple functionalities such as disease detection, AI assistance, and data storage into a single application, eliminating the need for separate tools.
4.2.2. Scalable Full-Stack Architecture: The platform is built using Next.js, allowing both frontend and backend operations within a unified framework, ensuring scalability and maintainability.
4.2.3. Real-Time Interaction: The system processes user inputs such as images and queries instantly and generates outputs without significant delay.

4.2.4 Modular Design: The architecture is divided into independent modules including disease detection, AI assistance, and database management, enabling easy upgrades and maintenance.
4.2.5. Efficient Data Handling: The system ensures proper storage and retrieval of user data, predictions, and interaction history using a NoSQL database.

4.3 METHODOLOGY

The methodology of the integrated AI platform is designed as a structured pipeline that ensures efficient processing of user inputs and generation of accurate outputs.
The process begins with user interaction through the frontend interface, where users can upload plant images or submit queries related to crop health.
The frontend is built using Next.js and provides a responsive and user-friendly interface.
Once the input is received, it is transmitted to the backend through API routes.
These API endpoints act as intermediaries that handle request validation, data formatting, and routing to appropriate modules.
  If the input is an image, the system forwards it to the disease detection pipeline, where the image undergoes preprocessing steps such as resizing, normalization, and format conversion.
The processed image is then passed to the CNN model for classification.
If the input is a textual query, it is directed to the AI assistance module, which processes the query using a dual-mode architecture.
The system first attempts to generate a response using a cloud-based AI model.
In case of connectivity issues, it switches to a locally deployed model, ensuring uninterrupted operation.
After processing, the system generates outputs such as disease classification, treatment recommendations, and preventive measures.
These outputs are then formatted into a structured response.
All results are stored in the MongoDB database, enabling data persistence and retrieval.
This allows users to access previous results and supports system analytics.
Finally, the processed output is returned to the frontend and displayed to the user in a clear and understandable format.
This end-to-end workflow ensures seamless interaction between all system components.

4.4 SYSTEM ARCHITECTURE

The system architecture follows a layered and modular design to ensure efficient communication between components.
It consists of four main layers: frontend, backend, AI processing, and database.

This layered approach improves system organization, maintainability, and scalability by clearly separating responsibilities among components.
The frontend layer handles user interaction and is responsible for collecting inputs and displaying outputs.
It is developed using Next.js, providing a responsive and user-friendly interface.
Users can upload plant images, submit queries, and view results such as disease predictions and advisory recommendations.
The backend layer acts as the core processing unit, managing API requests, processing incoming data, and routing it to appropriate modules.
It performs tasks such as request validation, data formatting, and communication with AI models and the database, ensuring efficient system operation.
The AI processing layer includes the CNN model for disease detection and the AI assistance module for generating recommendations.
The CNN model analyzes input images to classify plant diseases based on visual features, while the AI module generates advisory outputs such as treatment methods and preventive measures.
The system dynamically switches between cloud-based and local AI models to ensure continuous functionality.
The database layer, implemented using MongoDB, stores user inputs, prediction results, and interaction history, enabling data persistence and retrieval.

  The system ensures efficient data flow through RESTful APIs, where requests move from frontend to backend, then to AI modules, and finally to the database before returning responses to the user.
It also supports asynchronous processing to handle multiple requests simultaneously, ensuring real-time performance.
Security mechanisms such as input validation and error handling enhance reliability, while the modular design allows future enhancements such as IoT integration and advanced analytics without affecting existing functionality.
Fig. 4.1: Overall System Architecture of the Proposed AI-Based Plant Health Platform

4.5 IMPLEMENTATION

The implementation of the integrated AI platform is carried out using modern web and AI technologies.
The frontend is developed using Next.js, providing a responsive and interactive user interface.
The backend is implemented using API routes, which handle communication between the frontend, AI modules, and database.
These APIs ensure efficient request processing and data flow.
The CNN model is integrated into the system for image-based disease detection, while the AI assistance module is connected through APIs to generate advisory responses.
MongoDB is used for data storage, enabling efficient handling of structured and unstructured data.
The system ensures secure and efficient data management.
The modular implementation allows each component to function independently while maintaining seamless integration across the system.
The input interface for crop recommendation is shown in Fig.
4.2. Fig.
4.2: Crop Recommendation Input Interface The user provides parameters such as state, soil type, planting season, and harvest duration.
These inputs are used by the system to generate suitable crop recommendations.

4.6 RESULTS

The integrated AI platform successfully demonstrates the ability to combine multiple functionalities into a single system.
The platform processes user inputs efficiently and generates accurate outputs in real time.
The system ensures smooth communication between frontend, backend, AI modules, and database components.
The modular design supports scalability and allows future enhancements.
Additionally, the platform provides a user-friendly interface that simplifies interaction with complex AI functionalities.
Users can easily access features such as disease detection, crop recommendation, and AI assistance without technical expertise.
This improves usability and makes the system practical for real-world agricultural applications.
Overall, Objective 1 establishes a strong foundation for the proposed system by providing a unified, efficient, and scalable platform for plant disease detection and advisory support.
Fig. 4.3: System Home Page Interface

CHAPTER 5
OBJECTIVE 2: DEEP LEARNING-BASED DISEASE DETECTION

5.1 INTRODUCTION
Plant diseases significantly affect crop yield, quality, and overall agricultural productivity.
Early and accurate identification of diseases is essential to minimize losses and ensure effective treatment.
Traditional disease detection methods rely on manual inspection, which is often subjective, time-consuming, and dependent on expert knowledge.
This creates a need for automated systems capable of providing fast and reliable diagnosis.
In recent years, deep learning techniques, particularly Convolutional Neural Networks (CNNs), have shown remarkable performance in image classification tasks.
CNNs are capable of automatically learning hierarchical features such as edges, textures, and complex patterns directly from images without requiring manual feature extraction.
This makes them highly suitable for plant disease detection, where visual symptoms play a critical role.
In this project, a CNN-based model is developed to classify plant diseases from leaf images.
The model is trained using a large-scale dataset and integrated into the system to provide real-time predictions.

The objective is to create a robust and efficient model that can accurately identify diseases and support users with reliable diagnostic outputs.

5.2 KEY CONTRIBUTIONS

5.2.1 Deep Learning-Based Solution: The system replaces traditional manual diagnosis with an automated deep learning approach, improving accuracy and efficiency in disease detection.

5.2.2 High-Accuracy CNN Model: A well-structured CNN model is developed and trained on a large dataset, achieving high classification performance across multiple disease classes.

5.2.3 Large-Scale Dataset Utilization: The use of the PlantVillage dataset enables the model to learn diverse disease patterns and improve generalization.

5.2.4 Real-Time Prediction Capability: The trained model is optimized and integrated into the system to provide instant predictions when users upload images.

5.2.5 Scalable Model Design: The model architecture is designed to support future enhancements such as transfer learning, fine-tuning, and dataset expansion.

5.2.6 Seamless System Integration: The CNN model is embedded into the backend pipeline, ensuring smooth interaction with other modules such as AI assistance and database.

5.3 METHODOLOGY

The methodology for disease detection is implemented as a multi-stage pipeline consisting of data acquisition, preprocessing, model training, evaluation, and deployment.

5.3.1 Data Collection

The dataset is obtained from the PlantVillage dataset, which contains labeled images of plant leaves under different disease conditions.
The dataset includes both healthy and diseased samples, enabling the model to learn a wide range of visual patterns and disease characteristics.

5.3.2 Data Preprocessing

The raw images are preprocessed before training.
This includes resizing images to a fixed resolution, normalizing pixel values to a range of 0 to 1, and converting images into numerical arrays.
These steps ensure consistency in input data and improve model performance.

5.3.3 Dataset Splitting

The dataset is divided into training and validation sets.
The training set is used for learning model parameters, while the validation set is used to evaluate performance and prevent overfitting.

5.3.4 Model Design

A Convolutional Neural Network is designed using multiple layers such as convolutional layers, activation functions, pooling layers, and fully connected layers.
These layers work together to extract spatial features and classify plant diseases effectively.

5.3.5 Model Training

The model is trained using backpropagation, where the loss is minimized by updating weights using optimization algorithms such as Adam or SGD.
During training, performance metrics such as accuracy and loss are monitored to ensure proper learning.
Dropout techniques may be applied to reduce overfitting.

5.3.6 Model Evaluation

The trained model is evaluated using validation data.
Metrics such as accuracy and loss are used to assess performance and ensure the model generalizes well to unseen data.

5.3.7 Model Deployment

The trained model is saved in a suitable format (e.g., .h5) and integrated into the system.
During inference, user-uploaded images undergo preprocessing and are passed through the model to generate predictions.
The output is a probability distribution across classes, and the class with the highest probability is selected as the predicted disease.

5.4 DATASET DESCRIPTION

The dataset used in this project is the PlantVillage dataset, which is one of the most widely used datasets for plant disease classification.
It contains approximately 87,000 labeled images spanning 38 different classes, including both healthy and diseased plant leaves.
Each class represents a specific plant species and disease type, allowing the model to learn distinct patterns associated with different conditions.
The dataset includes images captured under controlled conditions, ensuring consistency in quality and labeling.
Before training, the dataset is organized into structured directories corresponding to each class.
Data augmentation techniques such as rotation, flipping, and zooming may be applied to increase dataset diversity and improve model generalization.
The dataset is divided into training and validation subsets, ensuring that the model is evaluated on unseen data.
This helps in assessing the model's ability to generalize to new inputs.

5.5 CNN MODEL ARCHITECTURE

The CNN model architecture is designed to effectively capture spatial features from leaf images.
It consists of a sequence of layers, each performing a specific function.
The input layer receives the preprocessed image.
This is followed by multiple convolutional layers, which apply filters to detect features such as edges, textures, and disease patterns.
Each convolutional layer is followed by an activation function, typically ReLU, which introduces non-linearity.
Pooling layers, such as max pooling, are used to reduce the spatial dimensions of feature maps, thereby decreasing computational complexity and preventing overfitting.
As the network deepens, it learns more abstract and high-level features.
The output from the convolutional layers is flattened and passed to fully connected layers, which perform classification.
The final layer uses a softmax activation function to produce probability scores for each class.

 The architecture is optimized to balance accuracy and computational efficiency, making it suitable for real- time applications.
The CNN architecture used for disease classification is illustrated in Fig.
5.1. Fig.
5.1: CNN Architecture for Plant Disease Classification

5.6 RESULTS

The CNN model demonstrates strong performance in classifying plant diseases, achieving high accuracy on the validation dataset.
The model is capable of distinguishing between multiple disease classes and healthy leaves with reliable precision.
When deployed in the system, the model processes user-uploaded images in real time and generates predictions within a short duration.
The output includes the predicted disease name along with a confidence score, which indicates the reliability of the prediction.
The integration of the model into the platform enables users to receive instant diagnostic results, improving decision-making and reducing dependency on manual inspection.
Fig. 5.2: Disease Detection Result with Treatment Recommendation The system provides detailed information about the detected disease along with confidence level and treatment recommendations.
This demonstrates the effectiveness of the model in generating actionable insights.

CHAPTER 6
OBJECTIVE 3: REAL-TIME AND OFFLINE AI SUPPORT

6.1 INTRODUCTION
This chapter focuses on the development of an intelligent AI-based assistance system capable of providing real-time agricultural guidance.
The objective is to enhance the system’s capability beyond disease detection by integrating advanced artificial intelligence models that can generate context-aware recommendations.
Farmers often require not only diagnosis but also actionable insights such as treatment methods, preventive measures, and crop suggestions.
To address this need, the system incorporates a dual-mode AI architecture that supports both online and offline environments.
The online mode uses a cloud-based model for advanced reasoning, while the offline mode uses a locally deployed model to ensure uninterrupted functionality.
This approach improves system reliability, accessibility, and usability, especially in rural areas with limited internet connectivity.

6.2 KEY CONTRIBUTIONS

6.2.1 Dual-Mode AI System: The system integrates both cloud-based and local AI models to ensure continuous operation under varying connectivity conditions.

6.2.2 Real-Time Advisory Generation: The system provides instant responses to user queries, enabling quick decision-making.

6.2.3 Context-Aware Recommendations: The AI models generate responses based on user inputs such as disease type, crop, and environmental conditions.

6.2.4 Offline Capability: The system ensures uninterrupted functionality by using a locally hosted model when internet access is unavailable.

6.2.5 Seamless Integration: The AI module is integrated with other system components such as disease detection and database for efficient workflow.

6.3 METHODOLOGY

The methodology of the AI assistance system is designed as a decision-based pipeline that dynamically selects the appropriate AI model based on system conditions.

6.3.1 Input Processing

User inputs such as queries or disease detection results are received through the frontend interface.
The input is processed and formatted into a structured prompt suitable for AI models.
This includes extracting relevant parameters such as disease name, crop type, and user query.

6.3.2 Prompt Engineering

The system generates a structured prompt to guide the AI model in producing accurate responses.
The prompt includes context such as disease information, symptoms, and required output format (e.g., causes, treatment, prevention).
This ensures consistency and relevance in generated responses.

6.3.3 AI Model Selection

The system checks internet connectivity and selects the appropriate model.
If internet access is available, the request is sent to the cloud-based AI model.
Otherwise, the system switches to the locally deployed model.
This decision-making process ensures reliability and adaptability.
Fig. 6.1: AI Model Selection and Response Flow

6.3.4 Response Generation

The selected AI model processes the input and generates a response.
The response includes detailed information such as disease causes, recommended treatments, preventive measures, and general agricultural advice.

6.3.5 Response Formatting

The generated output is formatted into a structured format for better readability.
This may include bullet points, categorized sections, and simplified language for user understanding.

6.3.6 Data Storage

The generated responses and user queries are stored in the database.
This enables retrieval of previous interactions and supports system analytics.

6.3.7 Output Delivery

The final response is sent back to the frontend and displayed to the user.
The system ensures minimal latency and smooth interaction.

6.4 GEMINI API INTEGRATION

6.4.1 Cloud-Based Processing
The system uses the Gemini API for generating intelligent responses when internet connectivity is available. This model provides high-quality, context-aware outputs.

6.4.2 Advanced Reasoning
The Gemini model is capable of understanding complex queries and generating detailed explanations, improving the quality of advisory responses.

6.4.3 Real-Time Communication
The API enables real-time communication between the system and the AI model, ensuring quick response generation.

6.4.4 Limitations
The cloud-based model depends on internet connectivity and may experience latency in low-network conditions.

6.5 OLLAMA (LLAMA 3.2) FALLBACK

6.5.1 Local Model Deployment
The system uses Ollama to run LLaMA 3.2 locally, enabling offline AI processing.

6.5.2 Offline Functionality
The local model ensures that the system continues to function even without internet access, making it suitable for rural environments.

6.5.3 Performance Considerations
 While local models may have slightly lower performance compared to cloud models, they provide reliable and consistent outputs.

6.5.4 Seamless Switching
The system automatically switches between cloud and local models without user intervention.

6.6 RESULTS
The AI assistance system successfully provides real-time and context-aware recommendations to users.
The dual-mode architecture ensures continuous operation in both online and offline environments.
The system demonstrates efficient integration of AI models with the overall platform, enabling smooth interaction and reliable output generation.
Users receive detailed and structured responses that assist in decision-making.

Fig. 6.2: AI Assistance Interface Showing User Query Fig.
6.3: AI Generated Advisory Response The figures demonstrate the complete interaction flow of the AI assistance module, where the user submits an agricultural query and the system generates a detailed advisory response.
The output includes structured recommendations such as disease detection methods, preventive measures, and treatment guidance.
This showcases the system's ability to provide accurate, context-aware, and real-time support to users.

CHAPTER 7
OBJECTIVE 4: MULTILINGUAL ACCESSIBILITY

7.1 INTRODUCTION
This chapter focuses on enhancing system accessibility by enabling multilingual interaction between users and the platform.
In agricultural environments, especially in rural areas, many users are more comfortable communicating in regional languages rather than English.
Language barriers often limit the adoption of digital technologies among farmers.
To address this issue, the proposed system incorporates multilingual capabilities that allow users to interact with the system in multiple Indian languages.
The system is designed to process user inputs and generate responses in the selected language, ensuring better understanding and usability.
This feature improves user engagement and makes the platform more inclusive and practical for real-world deployment.

7.2 KEY CONTRIBUTIONS

7.2.1 Multilingual Interaction Support: The system allows users to communicate in multiple Indian languages, improving accessibility.

7.2.2 Language-Aware Response Generation: The system generates responses in the user's preferred language, ensuring clarity and usability.

7.2.3 Integration with AI Assistance: The multilingual feature is integrated with the AI module to provide context-aware responses in different languages.

7.2.4 Improved User Experience: The system enhances usability for non-English-speaking users, making it suitable for rural environments.

7.2.5 Scalable Language Framework: The design allows additional languages to be integrated in future updates.

7.3 METHODOLOGY

The multilingual system is implemented using a structured pipeline that handles language selection, input processing, translation, and response generation.

7.3.1 Language Selection

The user selects the preferred language through the frontend interface.
This selection is stored and used throughout the interaction process to maintain consistency.

7.3.2 Input Processing

User inputs, whether text or query-based, are captured and processed.
If the input is in a regional language, it is identified and prepared for further processing.

7.3.3 Language Normalization

The system standardizes input text by converting it into a format suitable for processing.
This may include tokenization and encoding for compatibility with AI models.

7.3.4 AI-Based Response Generation

The processed input is passed to the AI assistance module, which generates responses based on the query.
The system ensures that the response aligns with the selected language context.

7.3.5 Translation Mechanism

If the AI model generates responses in a default language (e.g., English), the system translates the output into the user's preferred language using translation techniques.

7.3.6 Response Formatting

The translated output is formatted into a user-friendly structure, ensuring readability and clarity.

7.3.7 Output Delivery

The final response is delivered to the user in the selected language through the frontend interface, ensuring smooth interaction.

7.4 LANGUAGE SUPPORT SYSTEM

7.4.1 Supported Languages

The system supports multiple Indian languages, enabling users from different regions to interact effectively.

7.4.2 Language Detection

The system can detect the language of user input and process it accordingly.

7.4.3 Translation Integration

 Translation mechanisms are integrated to convert responses into the desired language.

7.4.4 Scalability
The system is designed to support additional languages in future enhancements without major modifications.

7.5 CHATBOT AND RESPONSE GENERATION

7.5.1 Conversational Interface
The system provides a chatbot-based interface for user interaction, enabling natural communication.

7.5.2 Context Awareness
The chatbot maintains context across interactions, ensuring relevant responses.
The working mechanism of the crop suggestion module is illustrated in Fig.
7.1. Fig.
7.1: Crop Suggestion System Working Overview The system considers multiple parameters such as location, soil type, seasonal conditions, and growth duration to generate suitable crop recommendations.
This ensures accurate and context-aware decision support for users.

7.5.3 Response Accuracy
The AI model generates accurate and informative responses based on user queries.

7.5.4 Real-Time Interaction
The chatbot provides instant responses, improving user experience.

7.6 RESULTS
The multilingual system successfully enables users to interact with the platform in their preferred language.
The system provides accurate and context-aware responses, improving accessibility and usability.
The integration of multilingual support enhances the overall effectiveness of the platform and makes it suitable for deployment in diverse agricultural environments.

CHAPTER 8
TOOLS AND TECHNOLOGIES

8.1 PLATFORM

8.1.1 Web-Based Platform: The proposed system is developed as a web-based application to ensure maximum accessibility and ease of use.
Users can access the platform through standard web browsers without requiring installation of additional software.

This is particularly beneficial for farmers, as it allows the system to be accessed from different devices such as desktops, laptops, and mobile phones.
The web-based approach ensures scalability and simplifies deployment.

8.1.2 Full-Stack Architecture: The system follows a full-stack architecture where both frontend and backend functionalities are integrated within a unified framework.
This reduces system complexity and improves performance.
The architecture ensures seamless communication between user interface, AI modules, and database components.

8.1.3 Cloud and Local Hybrid Execution: The platform supports both cloud-based and local execution.
Cloud services are used for advanced AI processing, while local execution ensures functionality in offline conditions.
This hybrid model enhances system reliability.

8.2 DEVELOPMENT TOOLS

8.2.1 Visual Studio Code: Visual Studio Code is used as the primary Integrated Development Environment (IDE) for writing and managing code.
It provides features such as debugging, syntax highlighting, extensions, and version control integration, which improve development efficiency.

8.2.2 Jupyter Notebook: Jupyter Notebook is used for developing and testing the CNN model. It allows step-by-step execution of code, visualization of outputs, and experimentation with different model configurations.

8.2.3 Git and GitHub: Git is used for version control, while GitHub is used for repository hosting. These tools enable tracking of code changes, collaboration, and maintaining different versions of the project.
It also ensures backup and proper project management.

8.2.4 Node.js Environment: Node.js is used to run the backend services and manage dependencies required for the Next.js framework.
It provides a runtime environment for executing JavaScript code outside the browser.

8.3 AI AND MACHINE LEARNING TOOLS

8.3.1 TensorFlow/Keras: TensorFlow and Keras are used to design and implement the CNN model. These frameworks provide pre-built layers, optimization algorithms, and training utilities required for deep learning applications.
They enable efficient model training and evaluation.

8.3.2 CNN Model (.h5): The trained CNN model is stored in .h5 format, which contains learned weights and

architecture.
This model is used during inference to classify plant diseases from input images.

8.3.3 Gemini API: The Gemini API is used for generating intelligent, context-aware responses. It enables the system to provide detailed advisory outputs such as treatment methods, causes of diseases, and preventive measures.

8.3.4 Ollama (LLaMA 3.2): Ollama is used to run the LLaMA 3.2 model locally. This ensures offline AI functionality, allowing the system to generate responses even without internet connectivity.
It improves system robustness.

8.3.5 Prompt Engineering Techniques: Structured prompts are used to guide AI models in generating accurate and relevant responses.
This ensures consistency in output format and improves the quality of advisory responses.

8.3.6 Data Processing Libraries: Libraries such as NumPy and Pandas are used for numerical operations, data preprocessing, and handling structured datasets.
These tools are essential for preparing data before training.

8.4 BACKEND AND DATABASE

8.4.1 Next.js Backend: The backend is implemented using Next.js API routes, which allow server-side processing within the same framework as the frontend.
This simplifies development and ensures efficient communication between components.

8.4.2 REST API Integration: RESTful APIs are used to connect different modules such as frontend, CNN model, and AI services.
These APIs handle data transfer, request processing, and response generation.

8.4.3 MongoDB Database: MongoDB is used as the primary database for storing user data, disease detection results, chatbot interactions, and system logs.
It is a NoSQL database that provides flexibility in handling unstructured data.

8.4.4 Data Storage and Retrieval: The database enables efficient storage and retrieval of historical data.
Users can view past disease detection results and AI responses, which improves usability and tracking.

8.4.5 Scalability and Performance: The backend system is designed to handle multiple user requests efficiently.
The use of asynchronous API calls ensures faster processing and better system performance.

8.5 INTEGRATION OF SYSTEM COMPONENTS

8.5.1 Frontend-Backend Integration: The frontend communicates with the backend using API requests.
User inputs such as images and queries are processed through these APIs.

8.5.2 AI Integration: The CNN model and AI assistance modules are integrated into the backend. The system dynamically routes requests to the appropriate module.

8.5.3 Data Flow Management: The system ensures smooth data flow between modules, including input processing, model prediction, AI response generation, and database storage.

CHAPTER 9
SOFTWARE TESTING AND RESULTS

9.1 INTRODUCTION
Software testing is an essential phase in system development to ensure that the application functions correctly, efficiently, and reliably under different conditions.
The proposed system integrates multiple components such as deep learning models, AI assistance, and a full-stack web platform, making testing crucial for validating system performance.
The testing process focuses on verifying individual modules, ensuring proper integration between components, and evaluating overall system behavior.
The results obtained from testing demonstrate the effectiveness of the system in real-world scenarios.

9.2 UNIT TESTING

9.2.1 Disease Detection Module: The CNN model is tested using sample leaf images to verify its ability to classify plant diseases accurately.
The model successfully predicts disease classes and provides confidence scores.

9.2.2 AI Assistance Module: The AI system is tested with various user queries related to plant health, treatment methods, and crop suggestions.
The system generates meaningful and context-aware responses.

9.2.3 Input Validation: The system is tested for different types of inputs such as valid images, invalid images, and empty inputs to ensure robustness and error handling.

9.2.4 Database Operations: The database is tested for storing and retrieving user data, prediction results, and interaction history.

9.3 INTEGRATION TESTING

9.3.1 Frontend-Backend Integration: The communication between the user interface and backend APIs is tested to ensure smooth data flow.

9.3.2 Model Integration: The CNN model is integrated with the backend, and its predictions are successfully displayed on the frontend.

9.3.3 AI Integration: The AI assistance module is integrated with the system, and responses are generated based on user inputs.

9.3.4 Data Storage Integration: The system correctly stores and retrieves data from MongoDB without errors.

9.4 SYSTEM TESTING

9.4.1 Functional Testing: The system is tested to ensure all functionalities such as disease detection, crop suggestion, and AI assistance work correctly.

9.4.2 Performance Testing: The system is evaluated for response time and efficiency. The platform provides quick results with minimal delay.

9.4.3 Usability Testing: The user interface is tested for ease of use, accessibility, and clarity of information.

9.4.4 Reliability Testing: The system is tested under different conditions to ensure stable performance.

9.5 RESULTS

The system successfully demonstrates accurate disease detection, effective AI assistance, and reliable crop recommendations.
The integration of multiple modules ensures seamless operation and user-friendly interaction.
Fig. 9.1: Disease Detection History Dashboard The system maintains a dashboard that stores previously analyzed plant images along with prediction details.
This enables users to track historical results and review past disease detections efficiently, demonstrating the system's data storage and retrieval capability.
 Fig.
9.2: Crop Recommendation Output The system generates a list of recommended crops based on user inputs such as location, soil type, seasonal conditions, and harvesting duration.
Each recommendation includes descriptive insights regarding suitability, yield potential, and environmental compatibility.
This demonstrates the effectiveness of the crop recommendation module in providing practical and data-driven agricultural guidance.

CHAPTER 10
CONCLUSION AND FUTURE WORK

10.1 CONCLUSION

The project titled "Deep Learning-Based Plant Health and Advisory Platform for Angiosperm Plants" presents an integrated and intelligent solution for modern agricultural challenges.
The system successfully combines deep learning techniques, artificial intelligence, and full-stack web development to provide a comprehensive platform for plant disease detection and advisory support.
The CNN-based disease detection module demonstrates the ability to accurately classify plant diseases from leaf images, reducing dependency on manual inspection and expert knowledge.
The use of the PlantVillage dataset enables the model to learn diverse disease patterns and achieve reliable performance.
In addition to disease detection, the integration of an AI assistance system enhances the functionality of the platform by providing detailed recommendations, including treatment methods and preventive measures.
The implementation of a dual-mode AI architecture ensures continuous operation by supporting both cloud-based and offline processing.
The multilingual feature improves accessibility and usability, allowing users to interact with the system in their preferred language.
This is particularly beneficial for farmers in rural areas who may not be comfortable using English-based applications.
The system is implemented as a full-stack web application using modern technologies such as Next.js and MongoDB, ensuring scalability, efficient data handling, and seamless user interaction.
The integration of multiple modules into a single platform provides a unified and user-friendly experience.
Overall, the proposed system achieves its objectives by delivering an accurate, efficient, and accessible solution for plant health monitoring and agricultural advisory.
It has the potential to improve crop productivity, reduce losses, and support data-driven decision-making in agriculture.
The system demonstrates the practical applicability of AI-driven solutions in precision agriculture.

10.2 FUTURE WORK AND CO-PO MAPPING

10.2.1 Model Improvement: Future work can focus on improving the accuracy of the CNN model by using advanced architectures such as ResNet, EfficientNet, or transfer learning techniques.

10.2.2 Real-World Dataset Expansion: The system can be enhanced by training the model on real-world field images to improve performance under varying environmental conditions.

10.2.3 Mobile Application Development: A mobile version of the system can be developed to increase accessibility and allow farmers to use the platform directly from smartphones.

10.2.4 Advanced AI Integration: The AI assistance module can be enhanced with more advanced reasoning capabilities and domain-specific knowledge for improved recommendations.

10.2.5 IoT Integration: Integration with IoT devices such as soil sensors and weather monitoring systems can provide real-time data for more accurate advisory.

10.2.6 Multilingual Expansion: Additional regional languages can be supported to further improve accessibility.

10.2.7 Performance Optimization: System performance can be improved by optimizing backend processing and reducing response time.

10.3 CO-PO MAPPING

Engineering Knowledge (PO1): Application of deep learning, AI, and web technologies demonstrates strong engineering fundamentals.
Problem Analysis (PO2): The system addresses real-world agricultural challenges such as disease detection and advisory support.
Design/Development of Solutions (PO3): A complete system is designed and implemented using modern technologies.
Modern Tool Usage (PO5): Tools such as TensorFlow, Next.js, and MongoDB are effectively utilized.
Communication (PO10): The system provides clear and understandable outputs to users.
Lifelong Learning (PO12): The project reflects continuous learning and application of emerging technologies.

REFERENCES

[1] S. P. Mohanty, D. P. Hughes, and M. Salathé, “Using Deep Learning for Image-Based Plant Disease Detection,” Frontiers in Plant Science, vol. 7, 2016.
[2] K. P. Ferentinos, “Deep Learning Models for Plant Disease Detection and Diagnosis,” Computers and Electronics in Agriculture, vol. 145, pp. 311–318, 2018.
[3] A. Kamilaris and F. X. Prenafeta-Boldú, “Deep Learning in Agriculture: A Survey,” Computers and Electronics in Agriculture, vol. 147, pp. 70–90, 2018.
[4] Y. LeCun, Y. Bengio, and G. Hinton, “Deep Learning,” Nature, vol. 521, pp. 436–444, 2015.
[5] Google DeepMind, “Gemini: A Multimodal AI Model,” 2024.
[6] Meta AI, “LLaMA: Open and Efficient Foundation Language Models,” 2024.
[7] A. Y. Ashurov et al., “Deep Learning-Based Plant Disease Detection Using CNN,” Frontiers in Plant Science, 2025.
[8] D. Devarajan et al., “AI-Based Real-Time Plant Disease Detection Using Enhanced CNN,” IEEE Access, 2026.
[9] V. Sharma et al., “Distributed Deep Learning for Plant Disease Detection,” Procedia Computer Science, 2024.
[10] M. Shafay et al., “Recent Advances in Plant Disease Detection Using Deep Learning Techniques,” 2025.